

Lesson Plan

More about function in JavaScript



Topics

- Arrow Function
- Recursive Function
- Function Chaining

Arrow Function

Introduction to Arrow Function

It is a feature which was introduced in ES6 version of javascript, which provides the clear and concise method of writing regular javascript functions in a more accurate and shorter way.

Advantage of using arrow function

1. Code size reduction

Arrow function provides a more concise syntax compared to the traditional function declaration, which reduces the overall size of the code.

2. Option return statement and braces

If we have a single expression then we can omit return and functional braces which will further reduce the code.

3. Improved readability

The arrow function syntax improves the code readability, making it easier for developers to understand and maintain the codebase.

Arrow function declaration

```
JavaScript
// arrow function declaration
const myFunction = (param1, param2) => {
  // logic code
};

// arrow function invoking
myFunction();
```

Let's have a detailed look at function declaration syntax-

1. myFunction

It is the variable name in which we are assigning our function, similar to that of function expression.

2. param1, param2

We can pass parameters inside the parenthesis, similar to that of function expression.

3. {}

Inside the block of curly braces, we will write our logic code which will be executed on function call.

4. myFunction()

It is used to invoke or call our arrow function.

Arrow function examples

Simple Example

```
JavaScript
// Arrow function
const greetUser = () => {
  console.log("Good Morning");
};

greetUser();

// Output

// Good Morning
```

Arrow function with one statement

If our function has only one statement then we do not need to write our single statement inside the curly braces.

```
JavaScript
// Arrow function
const greetUser = () => console.log("Good Morning");

greetUser();

// Output
// Good Morning
```

Arrow function with one parameter

```
JavaScript
// Arrow function
const greetUser = (name) => console.log("Good Morning", name);
greetUser("PW Skills");

// Output
// Good Morning PW Skills
```

We can also omit the parentheses around the parameter if our arrow function has only one parameter as given in below example.

```
JavaScript
// Arrow function
const greetUser = name => console.log("Good Morning", name);
greetUser("PW Skills");

// Output

// Good Morning PW Skills
```

Note

1. Similarly we can pass multiple parameters inside our arrow function as per our need, but keep in mind that we have to wrap them inside parentheses if they are more than one.
2. As we had accessed 'n number' of parameters in the regular function using the arguments keyword, that will not work with the arrow function.

Differences between the Arrow function and Regular function

Here are a few basic differences between these two as we will discuss more differences once we reach the advanced javascript section.

Arrow Function	Regular (Traditional) Function
• It has concise syntax using the '⇒' arrow operator, without the need for the function keyword. • It does not have its own arguments object, which provides the access to all the arguments passed to the function.	• It uses the function keyword followed by the function name. • It has its own arguments object, which provides the access to all the arguments passed to the function.

Recursive Function

Introduction to recursive function

A recursive function is a function that calls itself within its own body. Recursive function is quite useful when we can solve a problem that can be divided into smaller subproblems. By using recursive functions, we always try to break a complex problem into smaller and more manageable subproblems.

Before moving forward to example, we will have to know about two terms and its importance -

1. Base Case

Base case is the condition that determines when the recursive function should stop calling itself. It is the simplest form of the problem that can be directly solved without further recursion.

It also provides the termination condition for the recursive calls, if there will not be any base case then it will lead to infinite recursion. Also we have to make sure that base case should always be reached at a point else it will lead to infinite recursion.

2. Recursive Case

It represents the logic where the recursive function calls itself to solve the problem by breaking it down into smaller subproblems. It will be executed again and again until the base case is reached.

Example 1

In this example, we are getting a number as input and will have to print all the numbers smaller than that one by one. So if the number will be 3 then the expected output will be 3 2 1 as we are not including the zero in it.

```
JavaScript
function printNumber(number) {
  // base case
  if (number === 0) {
    return;
  }

  console.log(number);
  // recursive case
  return printNumber(number - 1);
}

printNumber(5);

/*
Output
5
4
3
2
1
*/
```

In the above example we can see that we had a base case which checks that the number is equal to zero, so that it can return and terminate the recursive call else it will be an infinite recursive call.

Also we are mentioning a recursive case which will call the function again by decreasing the value of number by one for the next iteration. This process will continue until the number will be equal to zero and the base case will be reached.

Example 2

Lets create a recursive function which returns the factorial of the given number.

The factorial of a number is the result of multiplying that number by all positive integers smaller than it down to 1. Also the factorial of zero will be 1, which is an edge case to which we have to remember.

JavaScript

```
function factorial(number) {  
  // base case  
  if (number === 0 || number === 1) {  
    return 1;  
  }  
  
  // recursive call  
  return number * factorial(number - 1);  
}  
  
const result = factorial(5);  
console.log(result);  
  
// Output  
// 120
```

Function Chaining

Introduction to function chaining

Functional or function chaining is a programming technique in JavaScript that allows us to call multiple functions one after the other on an object without the need for intermediate variables which makes our code cleaner and readable.

Benefits of using functional chaining

1. Readability

Functional chaining makes our code easier to read and understand, as we can see a sequence of actions applied to an object in a linear and clear way.

2. Conciseness

It reduces the need for creating intermediate variables, resulting in more concise code.

3. Method Chaining

Allows us to chain methods together when they return objects themselves. Will learn more while learning about array and its methods.

Example

```
let result = 0;

const calculator = {
  // function to add a number to result
  addNumber: function (num) {
    result += num;
    return this;
  },

  // function to subtract a number to result
  subtractNumber: function (num) {
    result -= num;
    return this;
  },

  // function to multiply a number to result
  multiplyNumber: function (num) {
    result *= num;
    return this;
  },

  // function to divide a number to result
  divideNumber: function (num) {
    result /= num;
    return this;
  },

  // function to print the value of result
  printResult: function () {
    console.log("Result is :", result);
    return this;
  },
};
```

```
const output = calculator

  .addNumber(10)

  .subtractNumber(5)

  .multiplyNumber(2)

  .divideNumber(2)

  .printResult();

// output
// Result is : 5
```

In the above code, we are returning “this” about which we will learn in future classes. As of now we should know that this keyword is nothing but the calculator object in itself to which you can verify by using console.log inside any of the above functions.

This is the reason because of which in definition we had seen that the function should return an object else it will throw an error.

So the equation becomes $0+10-5*2/2 = 5$ and then printing the value of result i.e 5.

Limitations of function chaining

1. Method return type

Chaining is the most effective way only when the object returns itself as we have seen in the above example but not every method will return the object so you cannot apply method chaining over there.

2. Overuse

For multiple nesting of the chaining it will be hard to read and understand the working of that code.

3. Debugging

In case of deep or multiple nesting, it becomes very hard to debug.

4. Implicit conversion

While using nesting there are chances where a method’s result implicitly changes itself into some other types which unintentionally modify the object’s state. So a user should be careful about the order of the method calls does not create undesirable results.